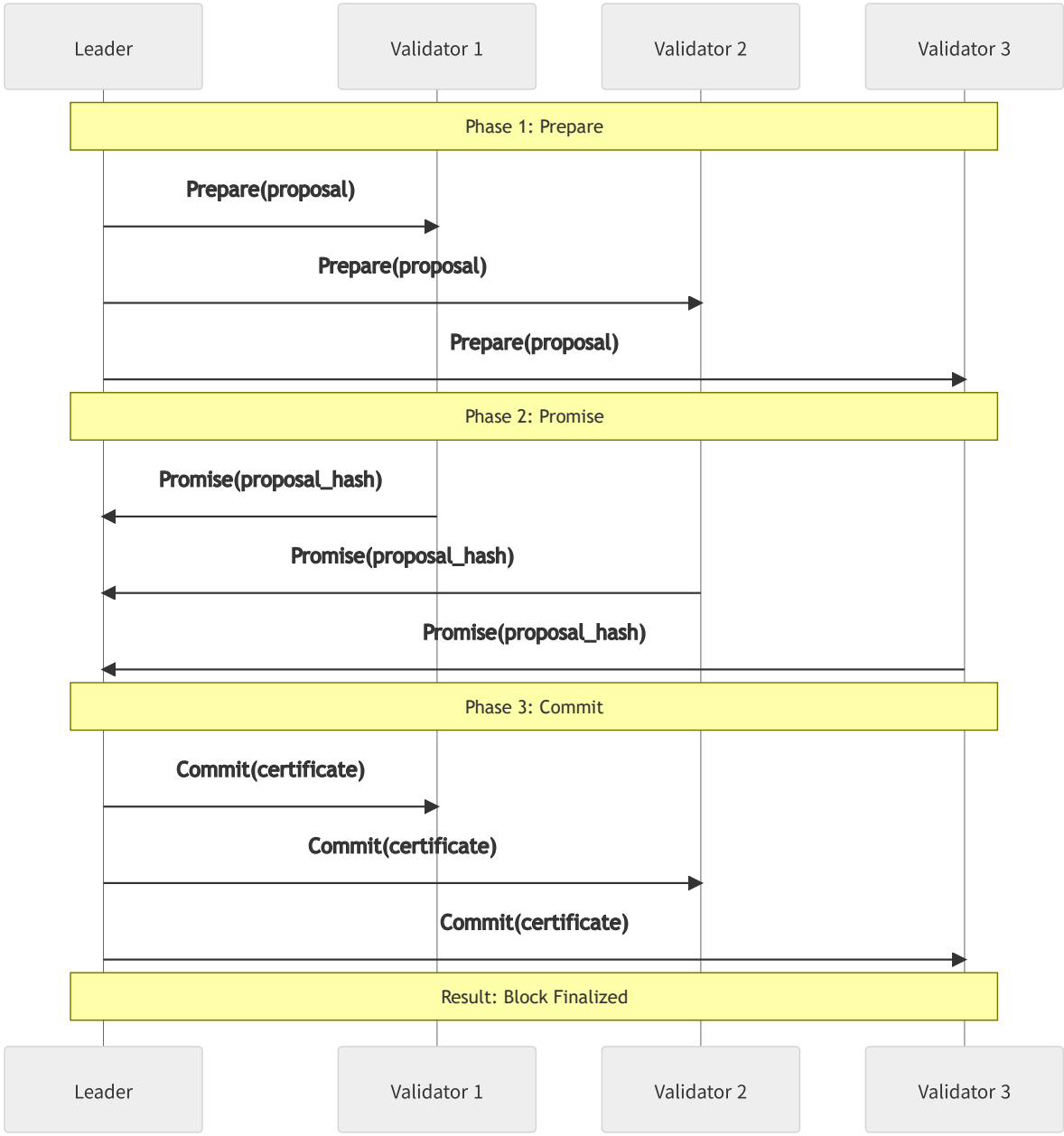
Slow Path 순수 구현

🛡️ Slow Path 핵심 개념

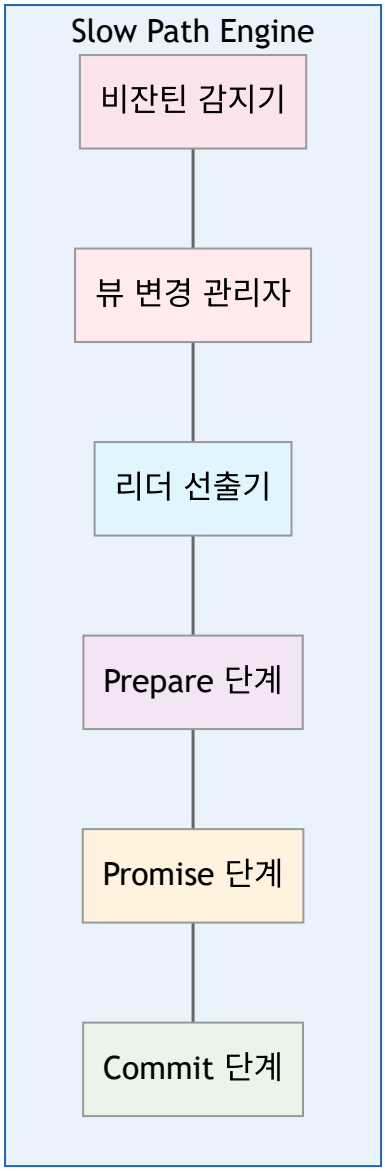
Slow Path는 Autobahn BFT의 **안전성 보장 엔진**으로, 네트워크 조건이 불안정하거나 비잔틴 노드가 감지될 때 **3f+1 노드로 안전한 합의**를 달성합니다.

🔄 3단계 합의 프로토콜

Slow Path는 전통적인 PBFT 스타일의 3단계 합의를 사용합니다:



🏗️ Slow Path 아키텍처



🔧 핵심 컴포넌트 구현

1. 리더 선출 시스템

```
/// Slow Path 리더 선출기 - 공정하고 결정론적인 리더 선출
pub struct SlowPathLeaderElection {
/// 현재 뷰
current_view: AtomicU64,
/// 노드 집합
```

```
node_set: NodeSet,
/// 리더 선출 알고리즘
election_algorithm: LeaderElectionAlgorithm,
/// 뷰 변경 히스토리
view_change_history: ViewChangeHistory,
}


impl SlowPathLeaderElection {
/// 현재 뷰의 리더 결정
pub fn get_leader_for_view(&self, view: u64) -> NodeId {
match self.election_algorithm {
LeaderElectionAlgorithm::RoundRobin => {
// 라운드 로빈 방식
let leader_index = (view as usize) % self.node_set.size();
self.node_set.get_node_by_index(leader_index)
}
LeaderElectionAlgorithm::WeightedRandom => {
// 가중 랜덤 방식 (스테이크 기반)
self.select_weighted_leader(view)
}
LeaderElectionAlgorithm::PerformanceBased => {
// 성능 기반 선출
self.select_performance_based_leader(view)
```

```
}

}

}

/// 성능 기반 리더 선출

fn select_performance_based_leader(&self, view: u64) -> NodeId {

let mut candidates: Vec<_> = self.node_set.nodes()

.iter()

.map(|node| {

let performance_score = self.calculate_performance_score(node);

(node.clone(), performance_score)

})

.collect();

// 성능 점수 기준 정렬

candidates.sort_by(|a, b| b.1.partial_cmp(&a.1).unwrap());

// 상위 후보들 중에서 뷰 기반 선택

let top_candidates = &candidates[..candidates.len().min(5)];

let selected_index = (view as usize) % top_candidates.len();

top_candidates[selected_index].0.clone()

}

/// 노드 성능 점수 계산

fn calculate_performance_score(&self, node: &NodeId) -> f64 {

let history = self.view_change_history.get_node_history(node);

// 성능 지표들
```

```
let uptime_score = history.uptime_percentage();
let latency_score = 1.0 / (1.0 + history.average_latency().as_secs_f64());
let reliability_score = history.success_rate();
// 가중 평균
0.4 * uptime_score + 0.3 * latency_score + 0.3 * reliability_score
}
}
```

2. Prepare 단계 구현

```
/// Prepare 단계 관리자
pub struct PreparePhaseManager {
/// 현재 뷰 상태
view_state: ViewState,
/// 제안 검증기
proposal_validator: ProposalValidator,
/// Prepare 메시지 생성기
prepare_generator: PrepareMessageGenerator,
/// 타임아웃 관리
prepare_timeout: TimeoutManager,
}

impl PreparePhaseManager {
```

```
/// Prepare 메시지 생성 (리더)
pub  async  fn  create_prepare_message(&mut  self, proposal: SlowPathProposal) -
// 1. 리더 권한 확인
if !self.is_current_leader() {
return Err(AutobahnError::InvalidState("Not the current leader".into()));
}
// 2. 제안 검증
self.proposal_validator.validate(&proposal).await?;
// 3. Prepare 메시지 생성
let  prepare_msg = PrepareMessage {
view: self.view_state.current_view(),
proposal: proposal.clone(),
leader: self.node_id(),
timestamp: SystemTime::now(),
// 리더의 암호학적 증명
leader_proof: self.generate_leader_proof(),
};
// 4. 메시지 서명
let  signed_prepare = self.sign_prepare_message(prepare_msg)?;
// 5. 타임아웃 설정
self.prepare_timeout.start_timeout(self.calculate_prepare_timeout());
Ok(signed_prepare)
}
```

```
/// Prepare 메시지 처리 (검증자)
pub  async  fn  handle_prepare_message(&mut  self, prepare: PrepareMessage) -> R
// 1. 기본 검증
self.validate_prepare_message(&prepare)?;
// 2. 리더 검증
if !self.verify_leader_authority(&prepare) {
return Err(AutobahnError::InvalidLeader("Invalid leader for view".into()));
}
// 3. 제안 내용 검증
let  validation_result = self.proposal_validator.validate(&prepare.proposal).awa
match  validation_result {
ValidationResult::Valid => {
// 4. Promise 메시지 생성
let  promise = self.create_promise_message(&prepare).await?;
Ok(Some(promise))
}
ValidationResult::Invalid(reason) => {
// 5. 뷰 변경 시작
self.initiate_view_change(ViewChangeReason::InvalidProposal(reason)).await?;
Ok(None)
}
}
}
```

```
/// Prepare 메시지 검증
fn validate_prepare_message(&self, prepare: &PrepareMessage) -> Result<(), Auto
// 1. 뷰 번호 검증
if prepare.view != self.view_state.current_view() {
return Err(AutobahnError::InvalidView("View mismatch".into()));
}
// 2. 서명 검증
if !self.verify_message_signature(prepare)? {
return Err(AutobahnError::InvalidSignature("Invalid prepare signature".into()));
}
// 3. 타임스탬프 검증
if self.is_message_too_old(prepare) || self.is_message_too_future(prepare) {
return Err(AutobahnError::InvalidTimestamp("Invalid prepare timestamp".into()));
}
// 4. 중복 검증
if self.has_received_prepare_for_view(prepare.view) {
return Err(AutobahnError::DuplicateMessage("Duplicate prepare message".into()));
}
Ok(())
}
}
```

3. Promise 단계 구현

```rust
/// Promise 단계 관리자
pub struct PromisePhaseManager {
/// Promise 메시지 수집기
promise_collector: PromiseCollector,
/// 임계값 계산기
threshold_calculator: ThresholdCalculator,
/// Promise 검증기
promise_validator: PromiseValidator,
/// 인증서 생성기
certificate_generator: CertificateGenerator,
}

impl PromisePhaseManager {
/// Promise 메시지 생성
pub async fn create_promise_message(&self, prepare: &PrepareMessage) -> Resul
let promise = PromiseMessage {
view: prepare.view,
prepare_hash: prepare.hash(),
promiser: self.node_id(),
timestamp: SystemTime::now(),
// 이전 뷰에서의 최고 제안 (있는 경우)
highest_prepared: self.get_highest_prepared_proposal(),
```

```
};

// 암호학적 서명

let signed_promise = self.sign_promise_message(promise)?;

Ok(signed_promise)

}

/// Promise 메시지 수집 및 처리 (리더)

pub async fn handle_promise_message(&mut self, promise: PromiseMessage) -> R

// 1. Promise 검증

self.promise_validator.validate(&promise)?;

// 2. Promise 수집

self.promise_collector.add_promise(promise.clone())?;

// 3. 임계값 확인 (2f+1)

let threshold = self.threshold_calculator.promise_threshold();

if self.promise_collector.count_for_view(promise.view) >= threshold {

// 4. Promise 인증서 생성

let promises = self.promise_collector.get_promises_for_view(promise.view);

let certificate = self.certificate_generator.create_promise_certificate(promise

// 5. 최고 제안 선택 (뷰 변경 시)

let selected_proposal = self.select_highest_proposal(&promises)?;

return Ok(PromiseResult::QuorumReached {

certificate,

selected_proposal,

});
```

```
}
Ok(PromiseResult::Continue)
}
/// 최고 제안 선택 (뷰 변경 시 안전성 보장)
fn select_highest_proposal(&self, promises: &[PromiseMessage]) -> Result<Option
let mut highest_view = 0;
let mut highest_proposal = None;
for promise in promises {
if let Some(ref prepared) = promise.highest_prepared {
if prepared.view > highest_view {
highest_view = prepared.view;
highest_proposal = Some(prepared.proposal.clone());
}
}
}
Ok(highest_proposal)
}
}

/// Promise 수집기
pub struct PromiseCollector {
/// 뷰별 Promise 저장소
promises_by_view: HashMap<u64, Vec<PromiseMessage>>,
```

```
/// 노드별 Promise 추적 (중복 방지)
promises_by_node: HashMap<(u64, NodeId), PromiseMessage>,
}


impl PromiseCollector {
/// Promise 추가
pub fn add_promise(&mut self, promise: PromiseMessage) -> Result<(), Autobahn
let key = (promise.view, promise.promiser.clone());
// 중복 Promise 체크
if self.promises_by_node.contains_key(&key) {
return Err(AutobahnError::DuplicateMessage("Duplicate promise from node".into())
}
// Promise 저장
self.promises_by_node.insert(key, promise.clone());
self.promises_by_view
.entry(promise.view)
.or_insert_with(Vec::new)
.push(promise);
Ok(())
}
/// 특정 뷰의 Promise 개수
pub fn count_for_view(&self, view: u64) -> usize {
self.promises_by_view.get(&view).map_or(0, |promises| promises.len())
```

```
}

/// 특정 뷰의 모든 Promise

pub fn get_promises_for_view(&self, view: u64) -> &[PromiseMessage] {

self.promises_by_view.get(&view).map_or(&[], |promises| promises)

}

}
```

4. Commit 단계 구현

```
/// Commit 단계 관리자

pub struct CommitPhaseManager {

/// Commit 메시지 수집기

commit_collector: CommitCollector,

/// 블록 완성기

block_finalizer: BlockFinalizer,

/// 상태 업데이터

state_updater: StateUpdater,

}


impl CommitPhaseManager {

/// Commit 메시지 생성 (리더)

pub async fn create_commit_message(&self, promise_certificate: PromiseCertifi

let commit = CommitMessage {
```

```
view: promise_certificate.view,

proposal_hash: proposal.hash(),

promise_certificate,

committer: self.node_id(),

timestamp: SystemTime::now(),

};

// 암호학적 서명

let signed_commit = self.sign_commit_message(commit)?;

Ok(signed_commit)

}

/// Commit 메시지 처리

pub async fn handle_commit_message(&mut self, commit: CommitMessage) -> Resu

// 1. Commit 검증

self.validate_commit_message(&commit)?;

// 2. Commit 수집

self.commit_collector.add_commit(commit.clone())?;

// 3. 임계값 확인 (2f+1)

let threshold = self.threshold_calculator.commit_threshold();

if self.commit_collector.count_for_view(commit.view) >= threshold {

// 4. 블록 완성

let commits = self.commit_collector.get_commits_for_view(commit.view);

let finalized_block = self.block_finalizer.finalize_block(commits, commit.propo

// 5. 상태 업데이트
```

```
self.state_updater.apply_block(&finalized_block).await?;

return Ok(CommitResult::BlockFinalized(finalized_block));

}

Ok(CommitResult::Continue)

}

/// Commit 메시지 검증

fn  validate_commit_message(&self, commit: &CommitMessage) -> Result<(), Autobah

// 1. Promise 인증서 검증

if !self.verify_promise_certificate(&commit.promise_certificate)? {

return Err(AutobahnError::InvalidCertificate("Invalid promise certificate".into(

}

// 2. 뷰 일관성 검증

if  commit.view != commit.promise_certificate.view {

return Err(AutobahnError::InvalidView("View mismatch in commit".into()));

}

// 3. 서명 검증

if !self.verify_message_signature(commit)? {

return Err(AutobahnError::InvalidSignature("Invalid commit signature".into()));

}

Ok(())

}

}
```

```rust
/// 블록 완성기
pub struct BlockFinalizer {
/// 트랜잭션 실행기
transaction_executor: TransactionExecutor,
/// 상태 머클 트리
state_merkle_tree: MerkleTree,
/// 블록 생성기
block_generator: BlockGenerator,
}


impl BlockFinalizer {
/// 블록 완성
pub async fn finalize_block(&mut self, commits: &[CommitMessage], proposal_h
// 1. 제안 복원
let proposal = self.get_proposal_by_hash(proposal_hash)?;
// 2. 트랜잭션 실행
let execution_results = self.transaction_executor.execute_transactions(&proposa
// 3. 상태 루트 계산
let new_state_root = self.state_merkle_tree.calculate_root_after_execution(&exe
// 4. 블록 생성
let block = self.block_generator.create_block(
proposal,
execution_results,
```

```
new_state_root,
commits.to_vec(),
)?;
// 5. 블록 검증
self.validate_finalized_block(&block)?;
Ok(block)
}
}
```

5. 뷰 변경 메커니즘

```
/// 뷰 변경 관리자 - 리더 실패 시 안전한 복구
pub struct ViewChangeManager {
/// 현재 뷰 상태
view_state: ViewState,
/// 뷰 변경 타이머
view_change_timer: ViewChangeTimer,
/// 뷰 변경 수집기
view_change_collector: ViewChangeCollector,
/// 새로운 뷰 생성기
new_view_generator: NewViewGenerator,
}
```

```rust
impl ViewChangeManager {
/// 뷰 변경 시작
pub  async  fn  initiate_view_change(&mut  self, reason: ViewChangeReason) -> Re
let  new_view = self.view_state.next_view();
// 1. 뷰 변경 메시지 생성
let  view_change_msg = ViewChangeMessage {
new_view,
old_view: self.view_state.current_view(),
reason: reason.clone(),
node: self.node_id(),
timestamp: SystemTime::now(),
// 현재 상태 증명
prepared_proposals: self.get_prepared_proposals(),
committed_proposals: self.get_committed_proposals(),
};
// 2. 뷰 변경 메시지 서명 및 브로드캐스트
let  signed_msg = self.sign_view_change_message(view_change_msg)?;
self.broadcast_view_change(signed_msg.clone()).await?;
// 3. 자체 뷰 변경 메시지 처리
self.handle_view_change_message(signed_msg).await?;
// 4. 뷰 변경 타이머 시작
self.view_change_timer.start_timer(new_view, self.calculate_view_change_timeout(
```

```
Ok(())
}
/// 뷰 변경 메시지 처리
pub  async  fn  handle_view_change_message(&mut  self, view_change: ViewChangeMe
// 1. 뷰 변경 메시지 검증
self.validate_view_change_message(&view_change)?;
// 2. 뷰 변경 수집
self.view_change_collector.add_view_change(view_change.clone())?;
// 3. 임계값 확인 (f+1)
let  threshold = self.threshold_calculator.view_change_threshold();
if  self.view_change_collector.count_for_view(view_change.new_view) >= threshold
// 4. 새로운 뷰 리더인지 확인
if  self.is_new_view_leader(view_change.new_view) {
// 5. 새로운 뷰 메시지 생성
let  new_view_msg = self.create_new_view_message(view_change.new_view).await?;
self.broadcast_new_view(new_view_msg).await?;
return Ok(ViewChangeResult::NewViewLeader);
} else {
return Ok(ViewChangeResult::WaitingForNewView);
}
}
Ok(ViewChangeResult::Continue)
}
```

```
/// 새로운 뷰 메시지 생성

async fn create_new_view_message(&mut self, new_view: u64) -> Result<NewViewM

let view_changes = self.view_change_collector.get_view_changes_for_view(new_vie

// 1. 안전한 제안 선택

let safe_proposal = self.select_safe_proposal(&view_changes)?;

// 2. 새로운 뷰 메시지 생성

let new_view_msg = NewViewMessage {

new_view,

view_changes: view_changes.to_vec(),

selected_proposal: safe_proposal,

new_leader: self.node_id(),

timestamp: SystemTime::now(),

};

// 3. 메시지 서명

let signed_msg = self.sign_new_view_message(new_view_msg)?;

Ok(signed_msg)

}

/// 안전한 제안 선택 (뷰 변경 시 안전성 보장)

fn select_safe_proposal(&self, view_changes: &[ViewChangeMessage]) -> Result<Op

let mut highest_view = 0;

let mut safe_proposal = None;

// 모든 뷰 변경 메시지에서 가장 높은 뷰의 준비된 제안 찾기

for view_change in view_changes {
```

```
for prepared in &view_change.prepared_proposals {
if prepared.view > highest_view {
highest_view = prepared.view;
safe_proposal = Some(prepared.proposal.clone());
}
}
}
Ok(safe_proposal)
}
/// 새로운 뷰 메시지 처리
pub async fn handle_new_view_message(&mut self, new_view: NewViewMessage) ->
// 1. 새로운 뷰 메시지 검증
self.validate_new_view_message(&new_view)?;
// 2. 뷰 전환
self.view_state.transition_to_view(new_view.new_view);
// 3. 타이머 정리
self.view_change_timer.clear_timers();
// 4. 새로운 뷰에서 합의 재시작
if let Some(proposal) = new_view.selected_proposal {
self.restart_consensus_with_proposal(proposal).await?;
} else {
self.restart_consensus_fresh().await?;
}
```

```
Ok(())
}
}
```

6. 비잔틴 노드 감지

```
/// 비잔틴 노드 감지기 - 악의적 행동 탐지 및 대응
pub struct ByzantineDetector {
/// 노드 행동 추적기
behavior_tracker: NodeBehaviorTracker,
/// 의심스러운 패턴 분석기
pattern_analyzer: SuspiciousPatternAnalyzer,
/// 비잔틴 노드 목록
byzantine_nodes: HashSet<NodeId>,
/// 감지 임계값
detection_thresholds: DetectionThresholds,
}

impl ByzantineDetector {
/// 메시지 검사 및 비잔틴 행동 감지
pub async fn analyze_message(&mut self, message: &ConsensusMessage, sender:
// 1. 기본 행동 추적
self.behavior_tracker.record_message(sender, message);
```

```
// 2. 즉시 감지 가능한 비잔틴 행동들
if  let Some(violation) = self.detect_immediate_violations(message, sender) {
self.mark_as_byzantine(sender.clone(), violation.clone());
return ByzantineAnalysisResult::ByzantineDetected(violation);
}
// 3. 패턴 기반 분석
if  let Some(suspicious_pattern) = self.pattern_analyzer.analyze_behavior(sender
if  suspicious_pattern.severity >= self.detection_thresholds.pattern_severity_th
let  violation = ByzantineViolation::SuspiciousPattern(suspicious_pattern);
self.mark_as_byzantine(sender.clone(), violation.clone());
return ByzantineAnalysisResult::ByzantineDetected(violation);
}
}
ByzantineAnalysisResult::Normal
}
/// 즉시 감지 가능한 비잔틴 행동
fn  detect_immediate_violations(&self, message: &ConsensusMessage, sender: &Node
match  message {
ConsensusMessage::FastVote(vote) => {
// 이중 투표 감지
if  self.behavior_tracker.has_conflicting_vote(sender, vote) {
return Some(ByzantineViolation::DoubleVoting {
round: vote.round,
```

```
conflicting_votes: self.behavior_tracker.get_conflicting_votes(sender, vote),
});
}
}
ConsensusMessage::SlowPrepare(prepare) => {
// 무효한 리더십 주장
if !self.verify_leadership_claim(sender, prepare.view) {
return Some(ByzantineViolation::InvalidLeadership {
view: prepare.view,
claimed_leader: sender.clone(),
});
}
}
ConsensusMessage::SlowPromise(promise) => {
// 모순된 Promise
if  let Some(conflicting_promise) = self.behavior_tracker.get_conflicting_promis
return Some(ByzantineViolation::ConflictingPromise {
view: promise.view,
promises: vec![promise.clone(), conflicting_promise],
});
}
}
_ => {}
```

```
}
None
}
/// 비잔틴 노드로 표시
fn mark_as_byzantine(&mut self, node: NodeId, violation: ByzantineViolation) {
self.byzantine_nodes.insert(node.clone());
// 로깅 및 알림
tracing::warn!(
"Byzantine behavior detected from node {}: {:?}",
node,
violation
);
// 네트워크에 비잔틴 노드 신고
self.report_byzantine_node(node, violation);
}
/// 노드가 비잔틴인지 확인
pub fn is_byzantine(&self, node: &NodeId) -> bool {
self.byzantine_nodes.contains(node)
}
/// 비잔틴 노드 목록 반환
pub fn get_byzantine_nodes(&self) -> &HashSet<NodeId> {
&self.byzantine_nodes
}
```

```rust
}

/// 노드 행동 추적기
pub struct NodeBehaviorTracker {
/// 노드별 투표 히스토리
vote_history: HashMap<NodeId, Vec<FastVote>>,
/// 노드별 Promise 히스토리
promise_history: HashMap<NodeId, Vec<PromiseMessage>>,
/// 노드별 메시지 타이밍
message_timing: HashMap<NodeId, Vec<MessageTiming>>,
/// 노드별 성능 지표
performance_metrics: HashMap<NodeId, NodePerformanceMetrics>,
}

impl NodeBehaviorTracker {
/// 메시지 기록
pub fn record_message(&mut self, sender: &NodeId, message: &ConsensusMessage)
let timing = MessageTiming {
timestamp: SystemTime::now(),
message_type: message.message_type(),
size: message.serialized_size(),
};
self.message_timing
```

```
.entry(sender.clone())
.or_insert_with(Vec::new)
.push(timing);
// 메시지 타입별 기록
match  message {
ConsensusMessage::FastVote(vote) => {
self.vote_history
.entry(sender.clone())
.or_insert_with(Vec::new)
.push(vote.clone());
}
ConsensusMessage::SlowPromise(promise) => {
self.promise_history
.entry(sender.clone())
.or_insert_with(Vec::new)
.push(promise.clone());
}
_ => {}
}
// 성능 지표 업데이트
self.update_performance_metrics(sender, message);
}
/// 충돌하는 투표 확인
```

```
pub fn has_conflicting_vote(&self, sender: &NodeId, vote: &FastVote) -> bool {
if let Some(votes) = self.vote_history.get(sender) {
votes.iter().any(|existing_vote| {
existing_vote.round == vote.round &&
existing_vote.proposal_hash != vote.proposal_hash
})
} else {
false
}
}
}
```

📊 Slow Path 성능 특성

예상 성능 지표

```
처리량: 30,000+ TPS
지연시간: 200-500ms (네트워크 조건에 따라)
CPU 사용률: 70-90%
메모리 사용량: 2-4GB (노드당)
네트워크 대역폭: 200-800 Mbps
비잔틴 허용: f개 노드 (전체의 1/3 미만)
```

안전성 보장

```
합의 안전성: 100% (수학적 증명)

활성성: 네트워크가 부분적으로 동기화된 경우 보장

비잔틴 허용성: 최대 f개의 악의적 노드 허용

뷰 변경: 리더 실패 시 자동 복구
```

🔍 모니터링 및 디버깅

```rust
/// Slow Path 성능 모니터링
pub struct SlowPathMonitor {
/// 단계별 지연시간 측정
phase_latency_tracker: PhaseLatencyTracker,
/// 뷰 변경 빈도 추적
view_change_tracker: ViewChangeTracker,
/// 비잔틴 감지 통계
byzantine_detection_stats: ByzantineDetectionStats,
}

impl SlowPathMonitor {
/// 실시간 성능 모니터링
pub async fn monitor_performance(&mut self) -> SlowPathMetrics {
SlowPathMetrics {
```

```
current_view: self.get_current_view(),
views_per_minute: self.calculate_views_per_minute(),
prepare_latency: self.phase_latency_tracker.average_prepare_latency(),
promise_latency: self.phase_latency_tracker.average_promise_latency(),
commit_latency: self.phase_latency_tracker.average_commit_latency(),
view_change_frequency: self.view_change_tracker.changes_per_hour(),
byzantine_nodes_detected: self.byzantine_detection_stats.total_detected(),
safety_violations: self.byzantine_detection_stats.safety_violations(),
}
}
}
```

🎯 결론

Pure Slow Path 구현은:

- ✅ **완전한 안전성**: 수학적으로 증명된 비잔틴 장애 허용
- ✅ **자동 복구**: 리더 실패 시 뷰 변경을 통한 자동 복구
- ✅ **비잔틴 감지**: 고급 패턴 분석을 통한 악의적 노드 감지
- ✅ **성능 최적화**: 3단계 프로토콜의 효율적 구현

다음 문서에서는 Autobahn DAG 관리자에 대해 알아보겠습니다.